

NestJS: Cuộc Cách Mạng Backend

Giải quyết bài toán "đống rác" mã nguồn bằng kiến trúc
chuẩn mực và TypeScript.

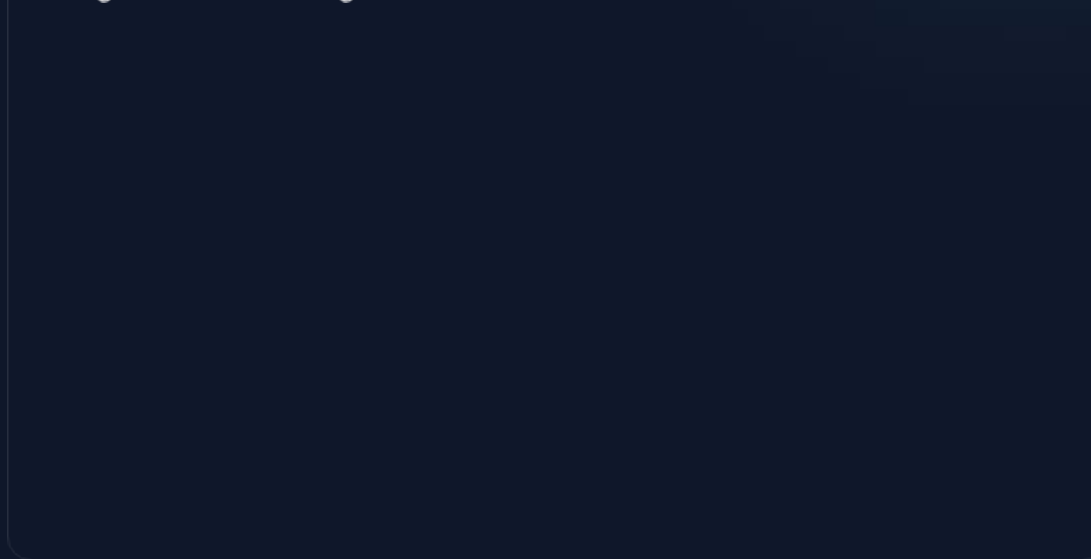
Tại sao dự án Backend hay thành "đống rác"?

⇒ **Sự tự do quá mức:** ExpressJS không có ràng buộc, dễ dẫn đến code tùy tiện, thiếu đồng nhất.

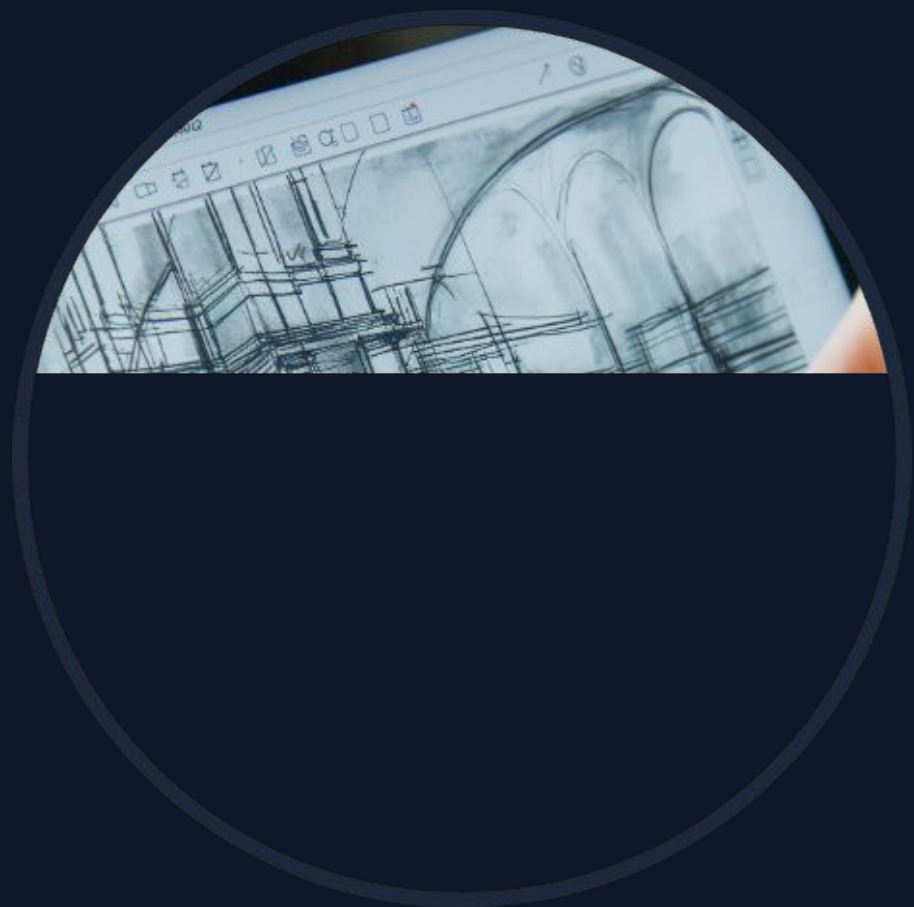
🔗 **Mất kiểm soát cấu trúc:** Khi dự án phình to, mã nguồn trở nên rối rắm như một cuộn len bị thắt nút.

👤 **Rào cản nhân sự:** Onboarding người mới cực kỳ khó khăn vì mỗi module được viết theo một kiểu khác nhau.

 Tangled wires vs organized tech



NestJS: Bản thiết kế kiến trúc hoàn chỉnh



Xây dựng trên nền tảng vững chắc

NestJS không "phát minh lại bánh xe". Nó sử dụng **ExpressJS** bên dưới và khoác lên một bộ khung kiến trúc (framework) mạnh mẽ.

Cung cấp một **nền móng vững chắc** để xây dựng những "tòa nhà chọc trời" phần mềm bền vững và dễ mở rộng.

Modules: Tổ chức mã nguồn thông minh



Tính đóng gói

Mỗi Module là một hộp chứa riêng biệt cho một tính năng cụ thể (Auth, User, Bookmark).



Decorator @Module

Sử dụng nhãn dán Decorator để định nghĩa rõ ràng các thành phần bên trong một hộp logic.



Dễ quản lý

Cấu trúc rõ ràng giúp toàn bộ team hiểu ngay dự án đang làm gì chỉ bằng cách nhìn vào sơ đồ module.

Controller: Người lễ tân sảnh khách sạn

Vai trò: Tiếp nhận và Trả lời



Tiếp nhận Request từ khách hàng (Client).



Điều hướng yêu cầu đến đúng bộ phận chuyên môn.



Tuyệt đối không tự xử lý logic, chỉ trả về kết quả cuối cùng.



Hotel receptionist

Service: Bộ não xử lý logic

Nơi thực hiện các công việc nặng nhọc và tính toán phức tạp nhất của ứng dụng.



Thực thi logic nghiệp vụ (Business Logic).



Tương tác trực tiếp với cơ sở dữ liệu.



Đảm bảo tính nhất quán và chính xác của dữ liệu.



DTO & Entities: Quản lý dữ liệu chặt chẽ



DTO (Data Transfer Object)

Người gác cổng dữ liệu:

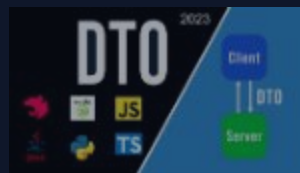
- Định nghĩa cấu trúc dữ liệu gửi lên.
- Lọc và kiểm tra (Validation) dữ liệu đầu vào.
- Đảm bảo hệ thống không nhận dữ liệu "rác".



Entities

Bản thiết kế lưu trữ:

- Đại diện cho cấu trúc bảng trong Database.
- Ánh xạ mã nguồn với cơ sở dữ liệu (ORM).
- Quy định cách dữ liệu được lưu trữ lâu dài.



Hành trình của một Request



1. Gửi Yêu Cầu

Client gửi POST /auth/signup kèm dữ liệu.



2. Controller

Tiếp nhận và chuyển cho Service tương ứng.



3. Service

Kiểm tra DTO, Hash pass và lưu vào DB qua Entity.



4. Phản Hồi

Kết quả được trả ngược lại cho người dùng.

SIX

Activity Network Diagram

1 Listing all tasks required for project completion.

Task Identification

2 Determining the order of tasks for efficiency.

Task Sequencing

3 Assigning resources to tasks based on needs.

Resource Allocation

Activity Network Diagram



Sức mạnh từ sự an toàn của TypeScript

100%

Kiểm soát lỗi tĩnh

Phát hiện lỗi ngay khi đang code thay vì chờ đến lúc sập server.

Tự động gợi ý code (Intellisense) cực mạnh.

✓✓ Refactor code an toàn, không lo hỏng hệ thống.

</> Code dễ đọc, dễ hiểu, tự tài liệu hóa.



So sánh ExpressJS và NestJS

Tiêu chí	ExpressJS	NestJS
Kiến trúc	Tự do (Unopinionated)	Chuẩn hóa (Opinionated)
TypeScript	Tùy chọn (Optional)	Mặc định (Default)
Tính mở rộng	Khó kiểm soát khi dự án lớn	Dễ dàng mở rộng (Scalable)
Doanh nghiệp	Phù hợp dự án nhỏ/vừa	Tiêu chuẩn cho dự án Enterprise

Giá trị cốt lõi cho Doanh nghiệp



Giảm chi phí

Bảo trì và sửa lỗi nhanh hơn nhờ cấu trúc tường minh và TypeScript.



Tăng tốc độ Team

Onboarding nhân sự mới chỉ tính bằng ngày thay vì bằng tháng.



Chất lượng cao

Code dễ viết Unit Test, đảm bảo tính ổn định tối đa cho hệ thống.